Reinforcement Learning
Lecture VIII: Temporal Difference

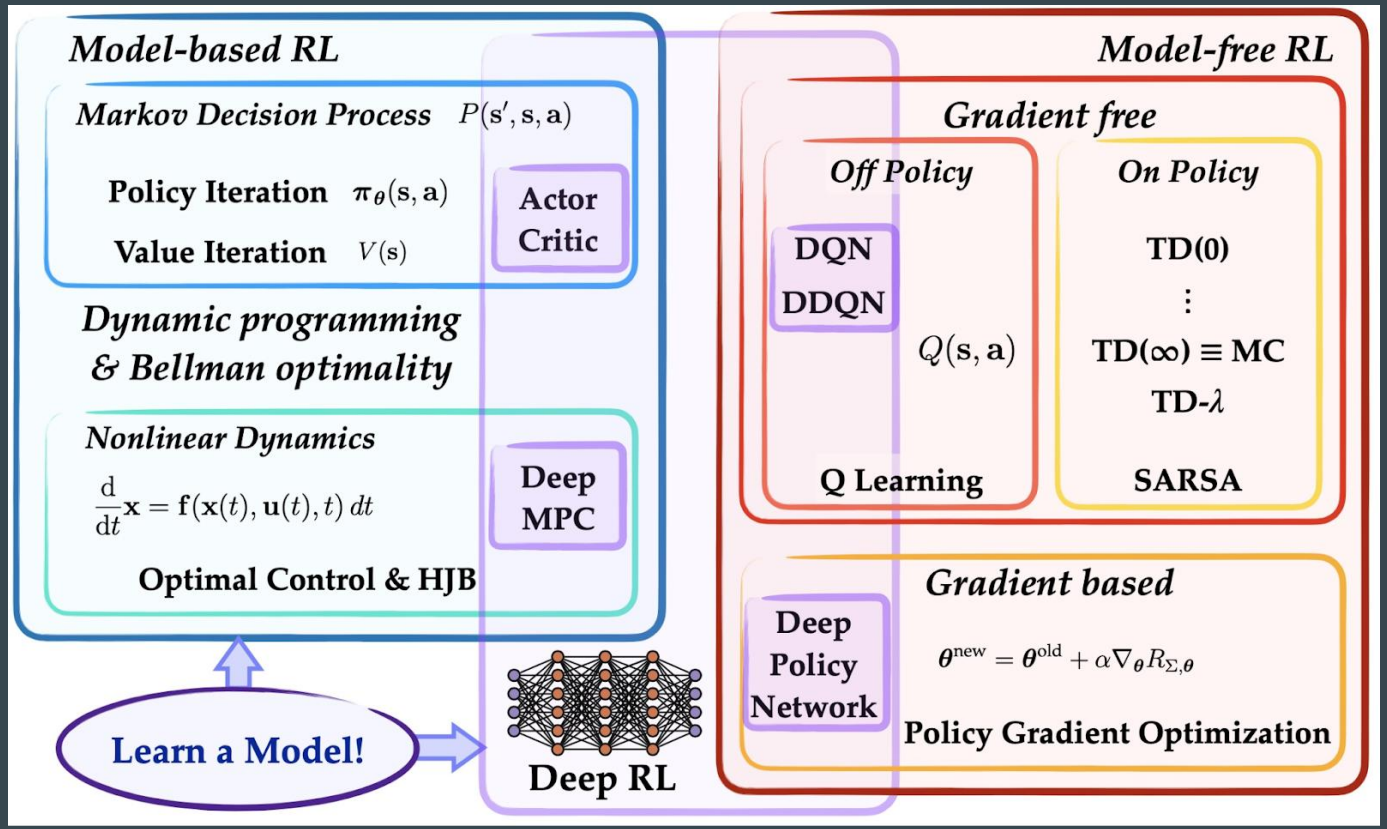
Asad Jeewa
University of Cape Town



What will be covered

- Temporal Difference Learning
- TD Control
- On-Policy vs Off-Policy RL

The Bigger Picture



Recall: Model-free RL

- Consider: MDP $(S, A, \boldsymbol{\gamma})$
  - Transition dynamics T and reward function R are unknown
- Goal: Find an optimal policy: π^*
- How: Learn from interaction with the environment (experience)
  - Monte Carlo methods
  - **Temporal Difference learning**

The Limitation of Waiting

- MC Bottleneck: Requires a complete episode to terminate before the value function can be updated.
- Complex/ real-world tasks:
  - Do not have a natural end point or terminal state.
  - Episodes may be extremely long.
- Waiting for termination is often inefficient or practically impossible.
- What can be done in these cases?

Real-world examples

- What real-world scenarios are better suited to model-free bootstrapping?
- Self-driving cars: Decisions update every millisecond from sensor data; waiting for a trip to end is impractical.
- Stock trading: Markets are continuous; portfolios adjust after each price change.
- Industrial robotics: Robots learn and optimise while production continues.
- Game AI: In Chess or Backgammon, agents evaluate moves immediately after the opponent responds.

TD Learning Properties

- Model-Free Algorithm: transition dynamics T and reward function R are unknown
- Bootstrapping: Updates estimates based on other estimates; no need to wait for the episode to end.
- Online Updates: Ideal for continuous or long tasks since it learns step-by-step.
- Data-Driven Learning: Learns directly from observed states, actions, and rewards.

Monte-Carlo vs Temporal Difference

MONTE-CARLO	TEMPORAL DIFFERENCE
Model-free	
Episodic domains	Non-episodic or continuing
Non-Markovian	Markovian
No Bootstrapping	Bootstrapping
Unbiased but high variance	Potentially biased but lower variance
Works only after episode ends	Faster convergence than MC.

TD is model-free bootstrapping of value estimates.

TD(0) Algorithm

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated
Algorithm parameter: step size $\alpha \in (0, 1]$
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\textit{terminal}) = 0$

Loop for each episode:
    Initialize S
    Loop for each step of episode:
        $A \leftarrow$ action given by π for S
        Take action A, observe R, S'
        $V(S) \leftarrow V(S) + \alpha\big[R + \gamma V(S') - V(S)\big]$
        $S \leftarrow S'$
    until S is terminal

TD Error

$$NewEstimate \leftarrow OldEstimate + LearningRate \times [Target - OldEstimate]$$

$$[R + \gamma V(s')] - V(s)$$

- $R + \gamma V(s')$ forms a more accurate guess than $V(s)$ because it contains one piece of real evidence
- TD Errors allows us to shift the current estimate $V(s)$ towards the target
- Represents the “difference" between our prediction and the actual observed outcome.
- Learning stabilizes once the difference between predictions and targets shrinks to zero.

What about α?

- The learning rate controls the impact of each update and is analogous to trust
- High α = high trust
  - Heavily prioritizes recent experience; quickly overwrites old knowledge.
- Low α = low trust
  - Skeptically averages new data with history to filter out noise
- Decaying α is analogous to step size $\frac{1}{N}$
  - Decaying trust starts at 1.0 and shrinks as N grows

From $n-step$ TD to Monte Carlo

- $TD(0)$ $(n = 1)$: Updates the current state estimate using only the immediate reward and the next state's value.
- $n-step\ TD$: Generalizes the update by looking ahead n steps before bootstrapping from a future state estimate.
- Monte Carlo $(n = \infty)$: As n reaches the end of the episode, the bootstrap term disappears and the update becomes the full return G_t
- How does n affect the bias-variance trade-off?

Bias-variance trade-off

- TD is biased: Values are updated using current estimates rather than final outcomes.
- MC has high variance: Updates rely on full trajectories, accumulating the noise of every random transition and action.
- Increasing n reduces bootstrapping bias but increases the variance of the update.

From Prediction to Control

- TD Prediction: estimate $V^{\pi}(s)$ for a fixed policy.
- Control: learn $Q(s, a)$ and improve the policy.
  - Generate episodes following an exploratory policy
  - Update $Q(s, a)$ at each step from TD errors
  - Improve the policy toward greediness with respect to Q.
  - Repeat until policy converges.

Seperating Policies

RL uses two distinct strategies

- Target policy is what you're learning
- Behaviour policy is how you're acting.
- On-Policy: Target policy is the same as behaviour policy
  - Learn from doing
- Off-Policy: Target policy is different from behaviour policy
  - Learn from watching

Separating Policies

- Why would we separate policies?
- Exploration: Act "crazily" (randomly) to find new rewards while keeping learned strategy stable.
- Data Efficiency: Reuse old data from past strategies to train current one.
- Safety: Train a high-risk autonomous car model by using data from a safe human driver

Choosing a Learning Approach

- Consider the intricacies of your task
  - Is safety is a priority?
  - Does the agent have to live with its mistakes?
  - Do you prefer stability?
  - Do you prefer speed?
  - Do you have limited data?

Choose On-Policy if the journey matters.

Choose Off-Policy if only the destination matters.

SARSA

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\mathit{terminal}, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal

Q-Learning

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\mathit{terminal}, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
    $S \leftarrow S'$
  until S is terminal

SARSA vs Q-Learning

Feature	SARSA (On-Policy)	Q-Learning (Off-Policy)
Behaviour Policy	$\epsilon -$ greedy	$\epsilon -$ greedy
Target Policy	Same as behaviour	Different: greedy policy
Update	$Q(s', a')$	$\max_a Q(s', a')$
Strategy	Learns from the action it actually takes.	Learns from the best action it could take.
Risk	Conservative: Avoids "accidental" bad outcomes.	Aggressive: Assumes it will always play perfectly.
Trade-off	More stable	More sample efficient

PRACTICAL TASK

- Use the code in the shared space to implement `sarsa.py`.
- Tip: Use the `q_learning.py` implementation to guide you. Also refer to the detailed algorithm in the lecture slides for guidance.

Resources

- Shared RL code space
- Sutton & Barto (1998), Reinforcement Learning: An Introduction, Chapter 6.
- https://www.datacamp.com/tutorial/introduction-q-learning-beginner-tutorial
- https://huggingface.co/learn/deep-rl-course/en/unit2/mc-vs-td